

- 7a) WAP to implement doubly link list with primitive operations
- Create a doubly linked list.
 - Insert a new node to the left of the node.
 - Delete the node based on a specific value.
 - Display the contents of the list.

PSEUDO CODE:

```
void createlist (int n) {
    struct node {
        struct Node *prev, *next;
        int data;
    };
}
```

```
void createlist (int n) {
    int i, data;
    struct node *newnode;
    for (i=1, i<=n; i++) {
        printf ("Enter data: ");
        scanf ("%d", &data);
        newnode = (struct node*) malloc (sizeof (struct node));
        newnode->data = data;
        newnode->prev = newnode->next = NULL;
        if (head == NULL)
            head = tail = newnode;
        else
            tail->next = newnode;
            newnode->prev = tail;
            tail = newnode;
    }
}
```



```
void InsertAtFront (int data) {
    struct node *newnode = (struct node*) malloc(sizeof(struct node));
    newnode->data = data;
    newnode->prev = NULL;
    newnode->next = head;
}
```

```
void InsertAtEnd (int data) {
    struct node *newnode = (struct node*) malloc(sizeof(struct node));
    newnode->data = data;
    newnode->next = NULL;
    newnode->prev = tail;
}
```

```
void deleteByValue (int value) {
    struct node *temp = head;
    if (head == NULL)
        return "empty list";
    while (temp != NULL && temp->data != value)
        temp = temp->next;
    if (temp == NULL)
        return "Element not found";
    if (temp == head)
        deleteAtFront();
    else if (temp == tail)
        deleteAtLast();
    else
        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;
    free(temp);
}
```


CODE:-

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    struct node *prev, *next;
    int data;
};

struct node *head = NULL, *tail = NULL;

void createlist (int n) {
    struct node *newnode;
    int i, data;
    for (i=1; i<=n; i++) {
        printf ("Enter data: ");
        scanf ("%d", &data);
        newnode = (struct node *) malloc (sizeof (struct node));
        newnode->data = data;
        newnode->prev = newnode->next = NULL;
        if (head == NULL) {
            head = tail = newnode;
        }
        else {
            tail->next = newnode;
            newnode->prev = tail;
            tail = newnode;
        }
    }
    printf ("Doubly linked list is created.");
}

void insertAtFront (int data) {
    struct node *newnode = (struct node *) malloc (sizeof (struct node));
    newnode->data = data;
    newnode->prev = NULL;
    newnode->next = head;
    if (head == NULL) {
```



```

    }
    else {
        head->prev = newnode;
        head = newnode;
    }
    printf("Node inserted at the beginning.");
}

void insertAtEnd (int data) {
    struct node *newnode = (struct node*) malloc (sizeof (struct node));
    newnode->data = data;
    newnode->next = NULL;
    newnode->prev = tail;
    if (tail == NULL) {
        head = tail = newnode;
    }
    else {
        tail->next = newnode;
        tail = newnode;
    }
    printf("Node inserted at the end.");
}

void deleteByValue (int value) {
    struct node *temp = head;
    if (head == NULL) {
        printf("List is empty. Create a linked list.");
        return;
    }
    while (temp != NULL && temp->data != value) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Value not found.");
    }
}

```



```

}
if (temp == head) {
    head = head->next;
    if (head == NULL) {
        tail = NULL;
    }
}

```

```

} else {
    head->prev = NULL;
}
free (temp);
}

```

```

else if (temp == tail) {
    tail = tail->prev;
    if (tail == NULL) {
        head = NULL;
    }
    else {
        tail->next = NULL;
    }
    free (temp);
}

```

```

} else {
    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
    free (temp);
}
}

```

```

void display () {
    struct node *temp = head;
    if (head == NULL) {
        printf ("list is empty.");
    }
    else {

```



```

printf("linked list: \n");
while (temp != NULL) {
    printf("%d -> ", temp->data);
    temp = temp->next;
}
}

int main() {
    int ch, n, value, data;
    do {
        printf("\n 1) Create doubly linked list  

        2) Insert at first 3) Insert at last  

        4) Delete by value 5) Display list");
        printf("\n Enter your choice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
                printf("\n Enter no. of nodes: ");
                scanf("%d", &n);
                createlist(n);
                break;
            case 2:
                printf("\n Enter data: ");
                scanf("%d", &data);
                insertAtFront(data);
                break;
            case 3:
                printf("\n Enter data: ");
                scanf("%d", &data);
                insertAtEnd(data);
                break;
            case 4:
                printf("\n Enter value: ");

```



```

        printf("%d", 4 value);
        deleteByValue (value);
        break;
    case 5:
        printf("%d", 5 value);
        display();
        break;
    default:
        printf("Invalid input");
}
while (ch != -1)
{
    // ...
}

```

Output:

1) Create doubly linked list 2) Insert at first 3) Insert at last 4) Delete by value 5) Display list.

Enter your choice: 1

Enter no. of nodes: 3

Enter data: 10

Enter data: 20

Enter data: 30

Doubly linked list is created

1) Create doubly linked list 2) Insert at first 3) Insert at last 4) Delete by value 5) Display list.

Enter your choice: 2

Enter data: 5

Node inserted at the beginning

1) Create doubly linked list 2) Insert at first 3) Insert at last 4) Delete by value 5) Display list

Enter your choice: 3

Enter data: 40

Node inserted at the end.

1) Create doubly linked list 2) Insert at first 3) Insert at last 4) Delete by value 5) Display list.

Enter your choice: 4

Enter value : 30

Deleted ele : 30.

1) Create doubly linked list 2) Insert at first 3) Insert at last 4) Delete by value 5) Display list.

Enter your choice : 5

Linked list :

5 → 10 → 20 → 40 → .

Q. 10